

Building a Low-Latency Limit Order Book:

A C++ Matching Engine and Performance Benchmark Study

Seoeun Baek

Stony Brook University

June 2026

Abstract

We implement a limit order book and matching engine first as a Python prototype, then port it to C++, in order to quantify the performance gap between the two for order-matching workloads. Using synthetic order flow generated from a uniform price and quantity distribution centered on a reference price, the C++ implementation achieves a $23.5\times$ throughput improvement over the Python prototype (822,870 vs. 34,990 orders/sec) and reduces p99 latency from $77.1\ \mu\text{s}$ to $2.25\ \mu\text{s}$. We additionally compare two C++ container choices for storing resting orders — a linear `std::vector` scan versus a price-sorted `std::map` of order queues — and find a modest 15% throughput improvement from the map-based structure, with no meaningful improvement in tail latency (p99/max) at this order volume and price-spread configuration. This study is limited to single-threaded execution on synthetic data and does not address concurrency, real exchange protocol overhead, or network latency.

1. Introduction

Order book performance is a first-order concern in electronic trading. A matching engine sits on the critical path between an incoming order and a filled trade, and differences in processing time on the order of microseconds can determine whether a resting order is captured or missed entirely. Firms that build and operate trading infrastructure therefore treat matching-engine latency as a primary engineering metric, not a secondary one.

Implementing a correct and reasonably fast matching engine is harder than it first appears. The core algorithmic problem — price-time priority matching — is simple to state: incoming orders match against the best available opposing price, and orders at the same price are filled in the order they arrived. The difficulty is in choosing data structures and an implementation language that keep this matching fast as the book grows, while preserving correctness under partial fills, order cancellation, and edge cases like zero-quantity orders.

This project follows a three-stage path: a Python prototype is built first to validate the matching logic, the same logic is then ported to C++, and the resulting implementation is benchmarked under synthetic order flow. A second C++ variant, using a price-sorted `std::map` in place of an unsorted `std::vector`, is built and benchmarked under the same conditions to isolate the effect of the underlying data structure from the effect of the implementation language.

The remainder of this report proceeds as follows. Section 2 gives brief background on limit order books and the rationale for using C++ in this domain. Section 3 describes the data model, matching algorithm, data structure choices, synthetic order generation, and benchmarking methodology. Section 4 reports throughput and latency results across all three implementations. Section 5 discusses what these results do and do not support. Section 6 states the study's limitations, and Section 7 concludes with directions for future work.

2. Background

A limit order book is the data structure an exchange uses to track outstanding buy and sell interest in an instrument. A limit order specifies a price and quantity; if it cannot be matched immediately, it rests in the book until a matching counter-order arrives or it is cancelled. The best bid is the highest resting buy price; the best ask (or offer) is the lowest resting sell price. The difference between them is the bid-ask spread. Most modern exchanges match orders under price-time priority: among orders at the same price, the order that arrived first is filled first.

C++ is the de facto language for latency-sensitive trading infrastructure, and the reasons are structural rather than incidental. C++ gives direct control over memory layout and allocation, which matters when an order book is processing tens of thousands of mutations per second and allocator overhead becomes visible in the latency profile. It compiles to native code with no interpreter or global lock standing between the code and the CPU, unlike Python's reference-counted, interpreter-bound execution model (notably including the GIL, which serializes bytecode execution even though it is not the dominant factor in the single-threaded comparison run here). The practical consequence, demonstrated empirically in Section 4, is an order-of-magnitude difference in both throughput and tail latency between otherwise identical matching logic written in the two languages.

3. System Design / Methodology

3.1 Order and Order Book Data Model

Each order is represented as a fixed structure with four fields: a unique integer id, a side (BUY or SELL, modeled as an enum class), a floating-point price, and an integer quantity. A companion Trade structure records the outcome of a successful match: the ids of the buy and sell orders involved, the execution price, and the matched quantity. Trade execution price follows the standard resting-order convention — a trade prints at the price of the order that was already in the book, not the price of the incoming order, since the resting order is the one whose price was time-stamped first.

3.2 Matching Algorithm

The matching engine implements price-time priority directly. When a new order arrives, the engine selects the opposite side of the book (incoming buys check resting sells, and vice versa) and walks the resting orders in arrival order, checking whether the incoming price crosses each resting price. A buy crosses a resting sell when the buy price is greater than or equal to the sell price; a sell crosses a resting buy under the mirrored condition. On a match, the engine fills the smaller of the two remaining quantities, decrements both orders accordingly, removes any order that reaches zero quantity, and continues until either the incoming order is fully filled or no further resting orders cross. Any unfilled remainder is appended to the book on the incoming order's own side.

3.3 Data Structure Choices

Two storage strategies for resting orders were implemented and compared. The baseline uses a single `std::vector<Order>` per side; matching requires a linear scan of this vector from the front, and a filled order is removed with `vector::erase`, which shifts all subsequent elements down by one position. This is the simplest correct

implementation, but its matching cost scales with the number of resting orders at prices the incoming order has not yet ruled out.

The alternative groups orders by price level using `std::map<double, std::vector<Order>>`, with one vector of orders per discrete price. `std::map` keeps its keys sorted automatically, so the best available price is always at one end of the structure: sell-side matching iterates forward from `begin()` (lowest price first), and buy-side matching iterates backward from `rbegin()` (highest price first). Once the incoming order's price no longer crosses the price at the current level, the scan terminates early rather than continuing through irrelevant levels, which is the structural advantage this design is intended to test.

3.4 Synthetic Order Flow Generation

No live exchange feed was used. Order flow was generated synthetically in both the Python and C++ implementations using matched random-number generation: side (buy or sell) drawn from a discrete uniform distribution over $\{0, 1\}$; price drawn from a continuous uniform distribution centered on a reference price of \$100.00 with a spread of $\pm \$5.00$; and quantity drawn from a discrete uniform distribution over $[1, 50]$ units. 10,000 orders were generated per benchmark run. This choice of distribution was made for simplicity and reproducibility rather than realism; it does not reproduce the clustering, burstiness, or fat-tailed arrival patterns of real order flow, a limitation discussed further in Section 6.

3.5 Benchmarking Methodology

All 10,000 orders were pre-generated before any timing began, so that random-number generation cost is excluded from the latency measurements. Each call to the matching function was individually timed using `std::chrono::high_resolution_clock` in C++ and `time.perf_counter()` in Python, both of which provide sub-microsecond resolution on the test machine. Per-order latencies were collected into a vector, sorted, and used to compute the mean and the 50th, 95th, and 99th percentiles, following standard practice for latency reporting in trading systems, where tail behavior is more operationally relevant than the average. All console output of individual trade fills was disabled during timed runs; an earlier run with trade-by-trade logging enabled showed a throughput reduction of roughly $6.5\times$ purely from I/O overhead, confirming that console output must be excluded from any latency-sensitive measurement. The C++ binaries were compiled with `g++` using the `-O2` optimization flag and the C++17 standard; all benchmarks were run on a MacBook Air (Apple Silicon, macOS).

4. Results

4.1 Python vs. C++ Baseline Comparison

Table 1 reports throughput and latency percentiles for the Python prototype and the `std::vector`-based C++ implementation, both processing the same 10,000-order synthetic stream. All timing figures are drawn from runs on the same machine.

Metric	Python	C++ (<code>std::vector</code>)	Ratio (Py / C++)
Throughput (orders/sec)	34,990	822,870	23.5×
Mean latency (μ s)	28.39	1.08	26.3×
p50 latency (μ s)	24.46	1.04	23.5×
p95 latency (μ s)	57.04	1.96	29.1×
p99 latency (μ s)	77.13	2.25	34.3×
Max latency (μ s)	455.67	32.83	13.9×

Table 1: Throughput and per-order latency, Python vs. C++ (`std::vector`), $n = 10,000$ orders.

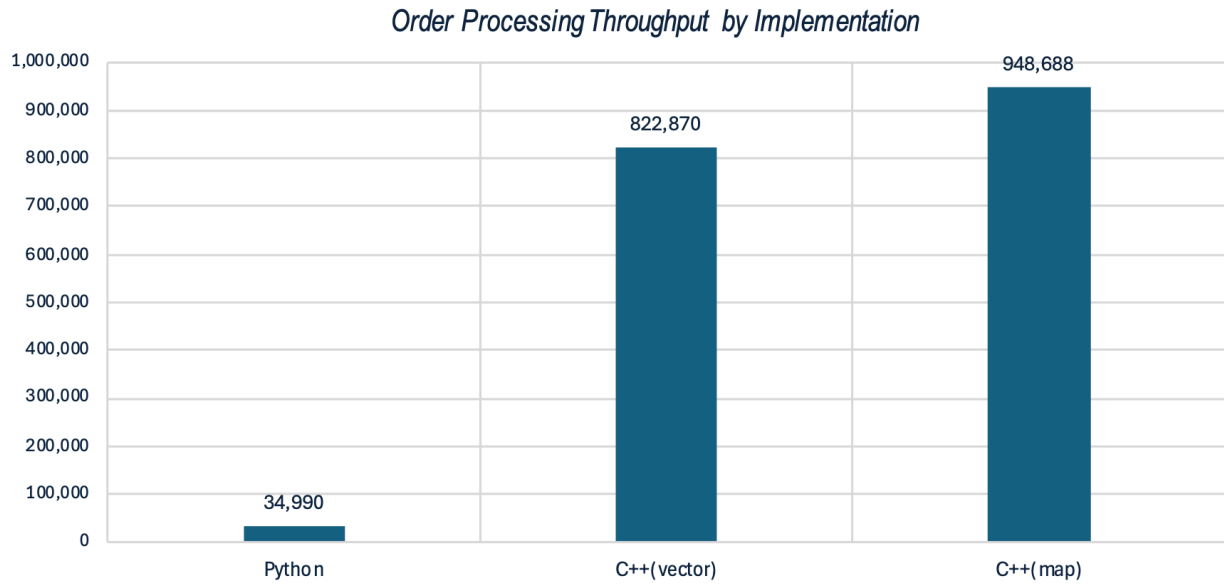


Figure 1: Order processing throughput by implementation.

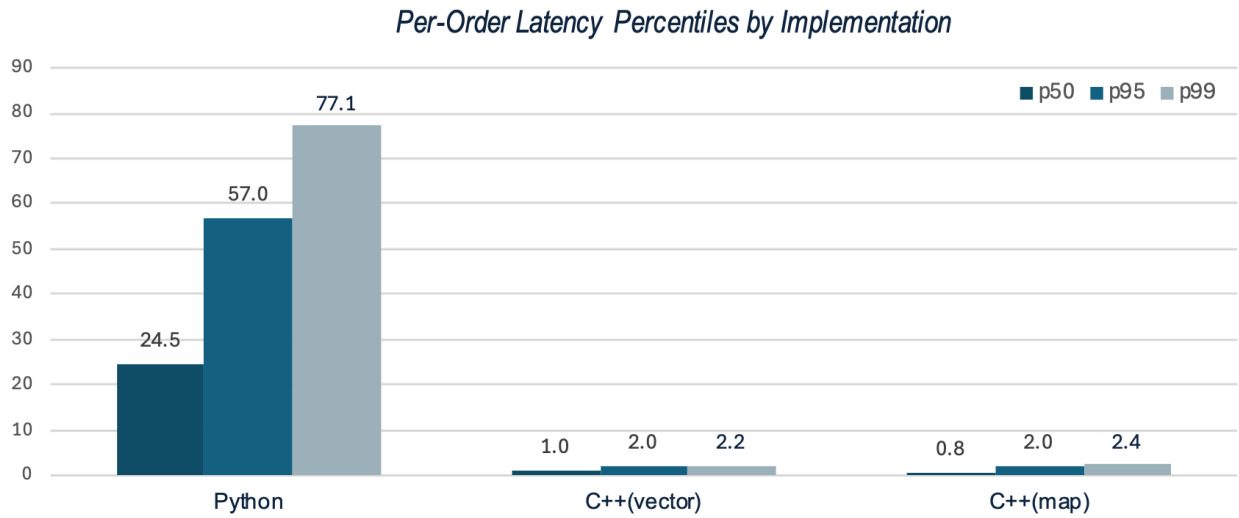


Figure 2: Per-order latency percentiles (p50/p95/p99) across all three implementations.

4.2 Data Structure Comparison Within C++

Table 2 compares the `std::vector` baseline against the `std::map`-based price-level implementation, both in C++, under the identical synthetic order stream.

Metric	C++ (<code>std::vector</code>)	C++ (<code>std::map</code>)	Change
Throughput (orders/sec)	822,870	948,688	+15.3%
Mean latency (μ s)	1.08	0.79	-27%
p50 latency (μ s)	1.04	0.83	-20%
p95 latency (μ s)	1.96	2.04	+4%
p99 latency (μ s)	2.25	2.38	+6%
Max latency (μ s)	32.83	32.04	-2%

Table 2: Throughput and latency, C++ `std::vector` vs. `std::map` implementations, $n = 10,000$ orders.

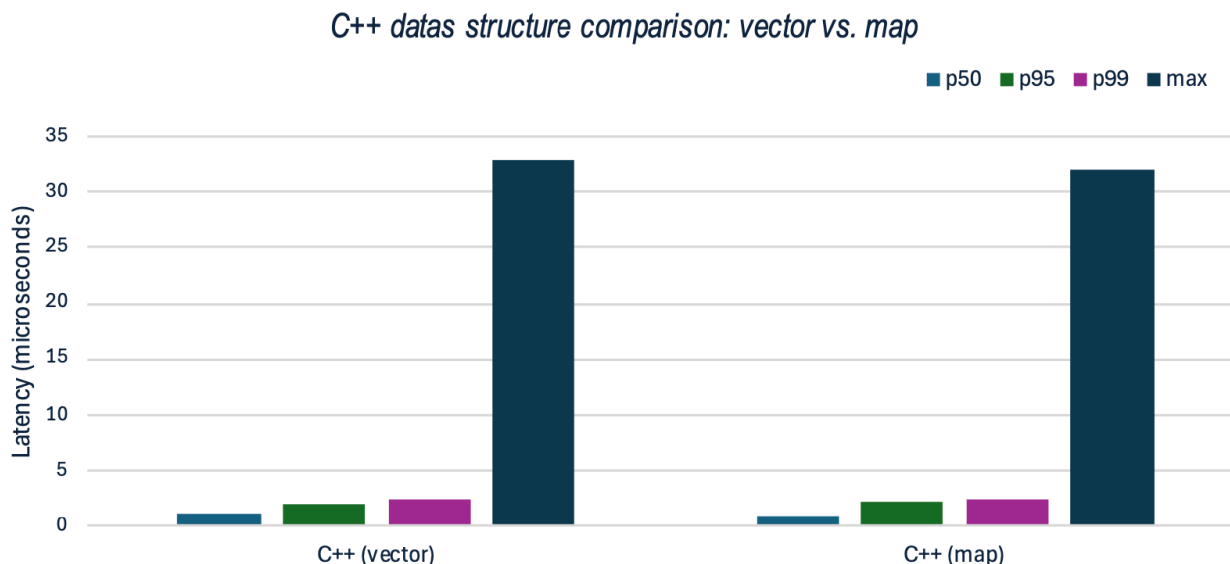


Figure 3: Latency percentiles and max latency, `std::vector` vs. `std::map` (C++ only).

4.3 Scaling Behavior

This study benchmarks a single order volume ($n = 10,000$) and a single price-spread configuration ($\pm \$5.00$ around a \$100 reference price). Scaling behavior — how throughput and tail latency change as order book depth and price dispersion increase — was not measured and is identified as a direction for future work in Section 7. The structural argument for `std::map` (early termination once the incoming price no longer crosses the current level) is expected to matter more as the number of non-crossable price levels grows; at the order volume and price concentration tested here, that advantage is visible in throughput but not yet in tail latency.

5. Discussion

The results support a clear, large effect of implementation language and a smaller, more ambiguous effect of data structure choice. The 23–34 $\times$ gap between Python and C++ across every latency percentile is consistent with the well-documented cost of Python's interpreter loop, dynamic typing, and per-object reference counting; none of these costs are present once the same algorithm is compiled and run as native C++ code. This result should not be read as a claim that Python is unsuitable for trading systems generally — it remains widely used for research, strategy development, and non-latency-critical infrastructure — only that it is a poor fit for the specific matching-engine hot path benchmarked here.

The `std::map` result is more nuanced and more interesting than a simple confirmation that ‘better data structures win’. Throughput improved by 15%, which is consistent with the map's early-termination property reducing wasted comparisons. However, p95 and p99 latency were not improved — they were marginally worse — and the maximum observed latency was essentially unchanged. A plausible explanation is that `std::map`'s tree-based structure trades away cache locality: `std::vector` keeps elements contiguous in memory, which is favorable for the CPU's cache prefetcher, while `std::map` nodes are individually heap-allocated and scattered, which can introduce pointer-chasing overhead that offsets some of the algorithmic savings. At the order volume and price concentration tested here (10,000 orders within a narrow $\pm \$5$ band), the number of distinct price levels per side was apparently not large enough for the map's early-termination advantage to dominate this cache-locality cost in every percentile.

Two points follow from this. First, the claim '`std::map` is faster than `std::vector` for an order book' is not unconditionally supported by this data; it is supported for mean and median latency, but not for the tail, under the specific synthetic conditions tested. Second, this is precisely the kind of result that benchmarking is supposed to surface — an intuitive design improvement that does not uniformly help in practice — and we report it without adjustment rather than only reporting the metric (median latency) that supports the expected conclusion.

6. Limitations

1. Synthetic data only.

Order flow was generated from a simple uniform distribution and does not reproduce the clustering, burstiness, or fat-tailed inter-arrival times of real exchange order flow. Conclusions about relative performance may not transfer directly to live market conditions.

2. Single-threaded execution.

All benchmarks ran on a single thread with no concurrency. Real trading infrastructure often involves concurrent order ingestion and matching, where lock contention and thread-safety overhead would materially change the comparison.

3. No network or I/O modeled.

Latency was measured purely at the in-process matching-function level. No network transport, exchange protocol parsing (e.g., FIX or ITCH), or disk I/O was included, all of which dominate real-world end-to-end latency budgets.

4. Hardware-specific results. All reported figures were measured on a single MacBook Air (Apple Silicon, macOS). Absolute latency and throughput numbers, and possibly the relative ranking of `std::vector` versus `std::map`, may differ on other CPU architectures, cache hierarchies, or under virtualization.

5. Single order-volume / price-spread configuration.

Only one order count (10,000) and one price dispersion setting were tested. The data-structure comparison in particular is sensitive to this choice, as discussed in Section 4.3.

7. Conclusion and Future Work

This project implemented a limit order book and matching engine under price-time priority, first in Python and then in C++, and benchmarked both alongside a second C++ variant using a price-sorted `std::map` in place of a linear `std::vector`. Three findings emerge. First, C++ outperforms an otherwise identical Python implementation by roughly 23–34× depending on the percentile, confirming that language choice dominates other design decisions for this workload. Second, swapping `std::vector` for `std::map` produced a real but partial improvement — higher throughput and lower median latency, but no improvement in tail latency — illustrating that an algorithmically favorable data structure does not automatically win on every metric. Third, console output proved to be a significant and easily overlooked confound in latency benchmarking, inflating measured throughput degradation by roughly 6.5× when left enabled during timed runs.

Four directions for future work follow directly from the limitations above:

1. Introduce lock-free or fine-grained-locked data structures and re-run the comparison under concurrent order submission, to test whether the `std::map` advantage changes under contention.
2. Replace synthetic uniform order flow with real historical tick data (e.g., ITCH or FIX message logs), to test whether the language and data-structure rankings hold under realistic arrival patterns and price clustering.
3. Vary order book depth and price dispersion systematically to characterize the scaling behavior only partially addressed in Section 4.3, and identify the order-volume threshold at which `std::map`'s early-termination advantage extends to tail latency.
4. Profile the `std::map` implementation at the cache-miss level (e.g., with `perf` or Instruments) to test the cache-locality hypothesis proposed in Section 5 directly, rather than inferring it from aggregate latency figures.

References

- [1] Stroustrup, B. The C++ Programming Language, 4th ed. Addison-Wesley, 2013.
- [2] Harris, T. “What Every Programmer Should Know About Memory.” (background reading on cache locality and data structure layout.)
- [3] cppreference.com. “`std::map`” and “`std::vector`” documentation. <https://en.cppreference.com/>
- [4] Aerospike. “What is P99 Latency?” <https://aerospike.com/blog/what-is-p99-latency/>
- [5] Wikipedia. “Central Limit Order Book.” https://en.wikipedia.org/wiki/Central_limit_order_book
- [6] Nasdaq. “TotalView-ITCH Market Data Specification.” <https://www.nasdaqtrader.com/>
- [7] Bouchaud, J.-P., Farmer, J. D., and Lillo, F. “How Markets Slowly Digest Changes in Supply and Demand.” Handbook of Financial Markets, 2009.
- [8] Meyers, S. Effective Modern C++. O'Reilly Media, 2014.